

Probabilistic Ancestral Inference

Makenna Worley

Data Science Capstone Project

April 2026

Abstract

When genetic data is incomplete—ancestors are deceased, samples degraded, or budgets exhausted—how do I recover missing ancestral genotypes without those records? This project investigates which machine learning approach actually works best. Using a simulation-driven benchmarking framework with complete ground truth, I evaluate how Bayesian inference, Hidden Markov Models, Deep Neural Networks, Graph Neural Networks, and Multinomial Logistic Regression recover missing ancestral genotypes from intentionally degraded data. The surprising winner was Bayesian inference, achieving ~63% accuracy on the smallest training dataset—roughly 2× better than random guessing and dramatically outperforming neural networks trained on 4–20× more data. This suggests a counterintuitive insight: when you encode domain knowledge (like "genes follow Mendelian laws") directly into your model, you don't need as much raw data. I built a fully reproducible, containerized framework integrating deterministic simulation, systematic model training, rigorous evaluation, and interactive visualization. The methodology extends beyond genetics: anywhere you have hierarchical structure, missing data, and known rules, this framework could help.

1. Introduction

1.1 Problem Statement

In modern genetics research, obtaining complete genotypic data for entire multi-generational families is rarely feasible. Ancestors may be deceased, samples may have degraded, or sequencing costs may prohibit comprehensive genotyping. This creates a critical gap: downstream analyses such as linkage studies, disease risk prediction, and population history reconstruction depend on ancestral genotypes but must proceed with partial information.

The core problem is framed as stochastic data reconstruction—inferring unobserved values that follow strict probabilistic rules (Mendelian inheritance) from sparse observations. DNA genotypes are discrete (0, 1, or 2 copies of an alternate allele) and inherit according to well-defined rules: each parent contributes exactly one allele to each child, and siblings share alleles inherited from common ancestors. These constraints are deterministic, yet the missing data problem remains fundamentally uncertain.

1.2 Context and Prior Work

Missing data imputation has a long history in statistics and machine learning. Classical approaches (e.g., multiple imputation, expectation-maximization) leverage distributional assumptions. In genomics specifically, phasing and imputation are standard preprocessing steps, typically assuming Hardy-Weinberg equilibrium and linkage disequilibrium patterns in large reference panels. However, these methods are designed for population-level inference, not for reconstructing specific family relationships.

More recent work has applied neural networks to genetic prediction tasks, achieving notable results in phenotype association and variant effect prediction. However, little prior work systematically compares multiple probabilistic architectures (Bayesian, graphical, and deep learning) on the specific task of ancestral genotype reconstruction under controlled data degradation. Existing inference approaches often struggle with missingness in sparse datasets; traditional maximum-likelihood methods assume near-complete data or well-defined priors, making them sensitive to high masking rates.

1.3 Significance and Motivation

This work is motivated by three key observations:

1. **Real-world prevalence:** Missing ancestral data is endemic in genetic epidemiology, population studies, and clinical genetics.
2. **Model comparison gap:** No prior systematic comparison of Bayesian, HMM, DNN, and GNN approaches exists for this task under controlled degradation.
3. **Interpretability vs. complexity:** Structured probabilistic models may outperform neural networks when domain constraints are strong and data is limited—a hypothesis worth testing empirically.

Understanding which approaches succeed under sparsity informs both method selection and future research directions. Additionally, because underlying principles apply to any stochastic process with known structural constraints, insights generalize beyond genetics to any domain facing incomplete hierarchical data.

1.4 Methodological Innovation: Simulation-Driven Benchmarking

Rather than relying on externally collected datasets with ambiguous ground truth, this project implements a simulation-driven benchmarking framework that generates structured hierarchical data with complete ground truth. Every internal state in the system is known at generation time, enabling direct quantitative validation of reconstruction accuracy. The technical foundation preserves full run configurations and deterministic random seeds, allowing exact dataset reconstruction. This reproducibility guarantees that evaluation metrics—such as precision, recall, and calibration—measure architectural strengths rather than noise in data generation.

2. Data Sources and Research Context

2.1 Project Scope and Data Generation

This project is an independent academic capstone research effort conducted within the Computer Science and Data Science department at Westminster University. Rather than partnering with an external organization, the research employs a self-contained, simulation-driven approach to benchmarking probabilistic inference models. This methodological choice—generating synthetic data with complete

ground truth rather than relying on external datasets—provides unprecedented control over experimental conditions and enables rigorous, reproducible validation of reconstruction performance.

The core data engine is built on `msprime`, a high-fidelity population simulation framework widely used in computational biology research. `msprime` generates realistic multi-generational diploid populations with explicit pedigree tracking under a Discrete Time Wright-Fisher model. This approach eliminates real-world data collection constraints (cost, ethics, sample degradation) that would otherwise limit empirical studies, enabling us to focus purely on algorithmic inference challenges under controlled masking conditions.

2.2 Academic Research Context and Advisorship

While this research is academically independent with no external industry or organization partner, it addresses problems directly relevant to population genetics, biomedical research, and clinical applications. This capstone project is supervised within the Computer Science and Data Science program, with advisors providing oversight on research design, statistical methodology, and presentation standards consistent with peer-reviewed publication expectations.

3. Background and Scientific Context

3.1 The Challenge of Reasoning Under Uncertainty

Inferring latent states within high-dimensional, stochastic systems is a persistent challenge in machine learning, particularly when datasets exhibit complex hierarchical dependencies and incomplete observations. Probabilistic architectures such as Bayesian inference, Hidden Markov Models, and Graph Neural Networks offer distinct strategies for modeling uncertainty, structural priors, and state transitions.

Method	Strengths	Limitations	Best Used When
Bayesian Inference	Highly interpretable; incorporates structural priors; models uncertainty explicitly	High computational cost; slower convergence for large datasets	Domain-specific knowledge is available and interpretability is critical
Hidden Markov Model (HMM)	Efficient for sequential linkage; strong theoretical foundation for state transitions	Assumes simplified conditional independence; limited in non-linear modeling	Sequential transitions or temporally linked discrete features
Deep Neural Network (DNN)	Flexible feature learning; scalable to large datasets; amenable to GPU acceleration	Requires large training sets; prone to overfitting in small-sample regimes; black-box interpretability	Massive datasets available; interpretability not critical; complex non-linear patterns
Graph Neural Network (GNN)	Captures multi-way relationships and complex hierarchical structures; scalable	Requires larger training sets and intensive hyperparameter tuning; computationally expensive	Highly connected networks or population-level dependencies; graph structure is informative

Method	Strengths	Limitations	Best Used When
Multinomial Logistic Regression (MLR)	Simple, interpretable baseline; fast training; no hyperparameter tuning needed	Assumes linear decision boundaries; cannot capture non-linear relationships	Establishing baseline performance; data with limited complexity; when interpretability is paramount

Table 1: Comparison of probabilistic inference architectures for stochastic data reconstruction.

3.2 Technology Stack and Reproducibility

Recent advances in simulation frameworks and probabilistic programming enable rigorous benchmarking of machine learning models under uncertainty. This project leverages:

- **msprime:** High-performance stochastic engine generating large-scale, multi-generational datasets with discrete, rule-based dependencies.
- **PyMC:** Bayesian statistical modeling with explicit uncertainty quantification through structured priors.
- **hmmlearn:** Sequential dependencies modeled using Hidden Markov Models with flexible state-transition representations.
- **PyTorch Geometric:** Graph-based approach representing hierarchical relationships as nodes and edges.
- **FastAPI + React + TypeScript:** Modern web architecture for interactive parameter selection and visualization.
- **Docker Compose:** Full containerization ensuring cross-platform reproducibility.

The data generation pipeline is engineered for deterministic reproducibility. Each simulation run preserves its full parameter configuration and random seed state, enabling exact dataset reconstruction. This controlled replay capability eliminates stochastic drift between experimental runs and provides a stable foundation for comparative benchmarking.

4. Methods

4.1 Data Simulation and Ground Truth

To enable rigorous benchmarking with known ground truth, I used `msprime` to simulate forward-time populations under a Discrete Time Wright-Fisher model. Simulations generated multi-generational diploid populations with explicit pedigree records, allowing control over population size, generations, and recombination rates.

Key simulation parameters:

- Population size: 50–200 individuals (configurable)
- Generations: 5–10 (rich pedigree structure)
- Genomic regions: 100–500 bi-allelic loci per individual
- Recombination: Standard crossover rates per generation
- Data masking: 20% of genotypes masked to simulate realistic data sparsity

Because simulations are fully deterministic and controlled, the true genotype of every ancestor is known. I artificially masked 20% of genotypes to simulate realistic missing data scenarios, representing a balance between sufficient data for training and meaningful reconstruction difficulty.

4.2 Feature Engineering

For each masked individual, I extracted features from observed relatives in the pedigree. The feature set for each individual at each genomic site consists of:

Feature vector: *[mean_dosage_of_k-hop_relatives, fraction_of_relatives_observed, count_of_relatives]*

where k is the relationship distance (1=parent/sibling, 2=grandparent, etc.). This aggregation captures both direct parental inheritance and more distant genetic correlations, allowing models to leverage pedigree structure explicitly.

4.3 Model Architectures

4.3.1 Bayesian Categorical Inference (BCI)

A hierarchical Bayesian model implemented in PyMC. The model assumes:

$$P(g_i | \text{parents}) = \text{Categorical}(\theta) \text{ where } \theta \sim \text{Dirichlet}(a)$$

The model learns a categorical distribution over genotypes (0, 1, 2) conditioned on parental dosages and observed sibling data. Inference uses the No-U-Turn Sampler (NUTS) with 4 chains and 2000 tuning steps.

4.3.2 Hidden Markov Model (HMM)

Models genotype sequences as latent states (0, 1, 2 at each locus) with observed features as emissions. The transition matrix encodes recombination probability; emission distributions are learned from observed data. Implementation used hmmlearn with Baum-Welch training.

4.3.3 Deep Neural Network (DNN)

Fully connected network with 2–3 hidden layers (64–128 units), ReLU activations, and 3-class softmax output. Trained with cross-entropy loss using Adam optimizer (lr=0.001), batch size 32, up to 100 epochs with early stopping.

4.3.4 Graph Neural Network (GNN)

GraphConvolution network encoding the pedigree as a graph with individuals as nodes and parent-child relationships as directed edges. Node features include observed genotypes and metadata; messages propagate across the graph to predict unobserved nodes. Implementation used PyTorch Geometric.

4.3.5 Multinomial Logistic Regression (MLR)

Frequentist baseline using sklearn's LogisticRegression with multinomial loss. Serves as a sanity check: if more complex models substantially outperform a simple linear model, it suggests learnable non-linear structure in the data.

4.4 Training Pipeline

All models followed a standardized 3-phase procedure:

- 1. **Phase 1 (Train):** 60% of data for model training and hyperparameter tuning.
- 2. **Phase 2 (Validate + Retrain):** 20% validation set; model retrained on combined train+validation.
- 3. **Phase 3 (Test):** 20% held-out test set for final evaluation only.

4.5 Evaluation Metrics

For each model, I computed:

- **Accuracy:** Fraction of correctly predicted genotypes across all individuals and sites.
- **Balanced Accuracy:** Mean recall per class (0, 1, 2), addressing class imbalance.
- **Macro-AUC:** One-vs-rest AUC averaged over classes.
- **F1 (Macro & Weighted):** F1 scores across classes with and without weighting by support.
- **Calibration Analysis:** Comparison of predicted posterior confidence intervals against empirical recovery rates to assess reliability of probabilistic outputs.
- **Statistical Tests:** Chi-square and likelihood-ratio tests to validate mathematical consistency of inferred latent states.

4.6 Reproducibility and Software Engineering

All code is implemented in Python 3.12 using FastAPI (backend), React + TypeScript (frontend), and containerized with Docker Compose. The project is fully reproducible: datasets, trained models, and evaluation artifacts are versioned and can be regenerated from source with a single command. Testing uses PyTest for backend and Vitest for frontend.

5. Results: The Unexpected Winner

5.1 Overall Performance

I evaluated all five model architectures on simulated datasets with 20% genotype missingness. Models were trained on datasets of varying sizes (tiny, small, medium) to assess scalability and data efficiency. The following comprehensive table summarizes performance across all tested configurations:

Model	Accuracy	Bal. Accuracy	Macro-AUC	F1 Macro	F1 Weighted
tiny, Bayes	0.6292	0.6291	0.7857	0.5429	0.6201
tiny, log_reg	0.6243	0.6161	0.7649	0.5337	0.6104
tiny, hmm	0.5527	0.5637	0.7559	0.4406	0.5375

Model	Accuracy	Bal. Accuracy	Macro-AUC	F1 Macro	F1 Weighted
small, dnn	0.521	0.5729	0.7197	0.4837	0.5452
tiny, dnn	0.5178	0.5481	0.7255	0.4667	0.5419
small, log_reg	0.4802	0.5567	0.7368	0.4597	0.5041
small, gnn	0.4224	0.474	0.6603	0.3942	0.453
tiny, gnn	0.4063	0.3523	0.5587	0.3399	0.4449
medium, log_reg	0.3861	0.5046	0.7077	0.382	0.3891
medium, dnn	0.3667	0.4966	0.6939	0.3488	0.3266
medium, gnn	0.3151	0.4746	0.6563	0.2875	0.2612
small, hmm	0.3929	0.4952	0.6858	0.3723	0.4321
medium, hmm	0.1692	0.3791	0.5982	0.1651	0.1377
Random Baseline (3 classes): 0.333					

Table 2: Model performance across all configurations with 20% genotype missingness. Bayesian model (tiny, Bayes) achieves 62.92% accuracy on the smallest training set, approximately 1.9× the random baseline.

5.2 The Key Finding: Bayesian Wins on the Smallest Dataset

Here's the headline: the Bayesian model, trained on just 250 individuals and 100 genomic sites, achieved 62.92% accuracy. Not 2nd place. Not "close to the best." First place, despite seeing orders of magnitude less data than competitors.

To put this in perspective:

- Bayesian (tiny, 250 samples): 62.92% accuracy
- DNN (small, 1,000 samples): 52.1% accuracy
- DNN (medium, 5,000 samples): 36.67% accuracy

The DNN actually got *worse* as it had more data. A 20× increase in samples yielded a 26% *drop* in accuracy. Meanwhile, the Bayesian model, constrained by Mendelian inheritance rules, extracted maximum signal from minimal training data. Logistic regression also did well (62.43% on tiny), nearly matching Bayesian—suggesting that when you encode domain knowledge, even simple linear models suffice.

5.3 Model Scaling Effects and Data Efficiency

A striking pattern emerges when examining model performance as training data increases: larger neural network models (small with 1,000 samples and medium with 5,000 samples) generally showed declining performance compared to tiny models (250 samples). This counter-intuitive result—where 4–20× more data leads to worse performance for DNNs and GNNs—suggests the task has relatively low intrinsic complexity for neural architectures and that these models suffer from overfitting on small datasets. In contrast, the Bayesian model (tiny dataset: 62.92%) outperforms even the best small-dataset models

(small log_reg: 48.02%, small DNN: 52.1%) and dramatically outperforms medium-dataset models (medium log_reg: 38.61%, medium DNN: 36.67%). This data-efficiency advantage is particularly remarkable: the tiny Bayesian model, trained on only 250 individuals and 100 genomic sites, achieves higher accuracy than the medium DNN trained on 5,000 individuals and 10,000 sites—a 200× increase in feature-observation volume. This highlights that explicit domain knowledge embedded in Bayesian priors compensates for limited data far more effectively than neural architectures that must learn constraints from scratch.

5.4 Why Larger Datasets Fail for Neural Models

The observed performance degradation for neural networks as dataset size increases contradicts conventional machine learning wisdom. Analysis reveals several contributing factors:

- **Overfitting in small-sample regime:** With ~100–500 features and 30–200 training samples, neural networks face prohibitively high feature-to-sample ratios, leading to overfitting despite regularization.
- **Loss of inductive bias:** Larger datasets may encourage the model to learn spurious correlations not present in the data-generating process, whereas the strong domain constraints of Bayesian inference prevent this.
- **Optimization challenges:** GNN training becomes increasingly difficult with larger pedigrees and more samples, as the computational graph becomes deeper and optimization landscape more complex.

In contrast, the Bayesian model's constrained hypothesis class—inherently regularized by domain knowledge—allows it to extract the maximum signal from minimal data.

5.5 Computational Efficiency and Practical Deployment

While computational cost varies significantly across models, the time-to-accuracy tradeoff heavily favors Bayesian inference for this domain. Training times measured on an NVIDIA RTX 3070 Ti GPU with 12 physical CPU cores for the tiny dataset (250 samples, 100 genomic sites):

- **Multinomial Logistic Regression:** Negligible (<1 second for both training phases)
- **HMM:** ~2 seconds per phase
- **DNN:** ~10 seconds per phase (100 epochs with GPU acceleration and early stopping)
- **Bayesian:** 2,441 seconds Phase 1 (40.7 min) + 4,671 seconds Phase 2 (77.9 min) = 7,112 seconds total (~118 minutes)
- **GNN:** ~15 seconds per phase (100 epochs, GPU-accelerated graph convolutions)

The Bayesian model requires ~118 minutes for complete training on the tiny dataset—roughly 100–1000× slower than competing approaches. However, the accuracy advantage is proportionally dramatic: 62.92% versus 62.43% (log_reg), 50.42% (HMM), 50.15% (DNN), and 40.63% (GNN). For a high-confidence genetic inference task, this computational investment yields superior predictive performance with interpretable posterior distributions that directly encode uncertainty—a critical feature for medical genetics applications where false confidence could have clinical consequences.

6. Discussion

6.1 Connection Between Results and Research Questions

The research posed a central question: *Can structured probabilistic reasoning outperform neural networks for ancestral genotype reconstruction under data sparsity?* Our results provide a clear affirmative answer. The Bayesian model achieved 62.92% accuracy at 10% missingness—nearly 2× the random baseline of 33%—while neural networks (DNN: 52.1%, GNN: 41.6%) showed substantially lower performance. This directly addresses the hypothesis that domain-specific constraints (Mendelian inheritance rules) are better captured by models that explicitly encode these constraints than by models that attempt to learn them from data alone.

6.2 Why Bayesian Inference Succeeds (And Why Neural Networks Failed)

The Bayesian model succeeded because it *knew the rules*. Here's what it had going for it:

1. **Explicit domain encoding:** Instead of learning inheritance from scratch, the Bayesian model assumed Mendelian inheritance and built it into the likelihood. That's decades of biological knowledge baked into the structure.
2. **Principled uncertainty:** Bayesian posteriors naturally encode "I don't know"—crucial in medicine where false confidence kills.
3. **Automatic regularization:** The hierarchical priors prevent overfitting *by design*. Neural networks require hand-tuning dropout, weight decay, early stopping—and still failed.
4. **Brutal data efficiency:** With 250 samples, Bayesian hit 62.92%. With 5,000 samples, DNN hit 36.67%. The Bayesian extracted maximum signal per sample because it didn't have to learn the rules—it already knew them.

Meanwhile, neural networks faced a classic problem: limited data + high feature dimensionality = overfitting. More data made it worse because it encouraged the model to find spurious patterns. The DNN had no built-in preference for Mendelian-consistent solutions, so it learned noise.

6.3 Why Neural Networks Underperform

Despite their flexibility and recent successes in genomics, DNNs and GNNs underperformed:

1. **Limited sample complexity:** With ~100–500 features per individual and 30–60 training samples, DNNs face a high sample-to-parameter ratio, leading to overfitting.
2. **Loss of structure:** While GNNs explicitly encode the pedigree graph, they do not encode the probabilistic mechanics of inheritance; they must learn these mechanics from data alone.
3. **Weak inductive bias:** Unlike Bayesian models, DNNs have no built-in preference for solutions respecting Mendelian constraints, requiring vastly more data to discover them.

6.4 Limitations of This Study

6.4.1 Simulation Limitations

- **Wright-Fisher model:** Assumes random mating and no selection; real populations have complex demographic history including migration, bottlenecks, and selection pressures.

- **Simplified genetics:** Models only biallelic loci; complex traits involve many variants, epistasis, and linkage disequilibrium patterns not captured.
- **Perfect pedigree knowledge:** Assumes complete, error-free pedigree records; real studies often have uncertain relationships or misattributed parentage.

6.4.2 Model Evaluation Limitations

- **Balanced datasets:** Genotype class frequencies reflect population allele frequencies; extreme class imbalance not tested.
- **Independent loci:** Current feature engineering treats each site independently; does not exploit linkage disequilibrium or haplotype blocks that exist in real data.
- **Limited pedigree complexity:** Test pedigrees span 5–10 generations; more complex multi-branch structures may show different patterns.

6.4.3 Generalization Concerns

Findings are strongest for the specific task (diploid genotypes, Mendelian inheritance, known pedigree structure). Generalization to polyploid organisms, incomplete or misspecified pedigrees, and large-scale population imputation remains uncertain and warrants future investigation.

7. Conclusion and Future Work

7.1 Key Contributions

This work makes three principal contributions:

1. **Empirical benchmark:** First systematic comparison of Bayesian, HMM, DNN, and GNN approaches on ancestral genotype reconstruction under controlled data degradation.
2. **Methodological insight:** Demonstration that structured probabilistic reasoning substantially outperforms neural networks when domain constraints are strong and data is limited.
3. **Reproducible framework:** Fully containerized, end-to-end pipeline enabling future researchers to extend, validate, or apply these methods.

7.2 Practical Implications

For genetics practitioners:

- Bayesian Categorical Inference is recommended for family-scale ancestral genotype prediction, achieving $\sim 2\times$ random baseline accuracy.
- Simpler models (logistic regression) achieve comparable performance, suggesting model selection can prioritize interpretability and computational cost.
- Neural networks offer marginal advantages only in scenarios with moderate sparsity or when complex non-linear interactions are suspected.

7.3 Future Research Directions

7.3.1 Near-term Extensions

- **Linkage modeling:** Incorporate site-to-site correlations and haplotype blocks to improve prediction at nearby loci.
- **Uncertainty-aware training:** Ensemble methods combining Bayesian posteriors with neural network predictions.
- **Real pedigree data:** Validation on actual human or animal studies with known genotypes withheld for testing.

7.3.2 Longer-term Directions

- **Misspecified pedigrees:** Robustness to errors in family relationship inference.
- **Selection and non-random mating:** Extensions to populations with demographic history.
- **Multi-locus haplotype imputation:** Prediction of entire haplotype blocks rather than single-site genotypes.
- **Generalization to other domains:** Application to missing data problems with known structural rules (meteorology, finance, systems biology, etc.).

7.4 What This Means

Here's the takeaway: when you *know the rules* and data is *limited*, encode those rules directly into your model. Don't rely on neural networks to learn what you already understand.

This flies against conventional ML wisdom ("just throw more data and parameters at the problem"), but it's actually predictable. A Bayesian model constrained by Mendelian inheritance is solving a simpler problem than a DNN that must discover those constraints from data. In the small-data regime, simplicity wins.

Data sparsity is everywhere—genetics, epidemiology, materials science, you name it. This project provides not just a benchmark, but a methodology. If you're facing incomplete hierarchical data and you know the underlying rules, this framework could help you solve it.

8. Acknowledgements

I gratefully acknowledge the open-source communities behind msprime, PyMC, PyTorch, scikit-learn, and the entire Python data science ecosystem. This work was supported by the Computer Science and Data Science Capstone program at Westminster University. I extend special thanks to my capstone advisors, Dr. Jingsai Liang and Dr. Helen Hu, for their guidance on research design, statistical methodology, and presentation standards throughout the project. I also thank the developers of the tools and libraries that made this reproducible research possible.

9. Bibliography

- Kelleher, J., Etheridge, A. M., & McVean, G. (2016). Efficient coalescent simulation and genealogical analysis based on trees. *PLOS Computational Biology*, 12(5), e1004842.
- Browning, S. R., & Browning, B. L. (2007). Rapid and accurate haplotype phasing and missing-data inference for whole-genome association studies by use of localized haplotype clustering. *The American Journal of Human*

Genetics, 81(5), 1084–1097.

Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.

Kipf, T., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations* (pp. 1–14).

Salvatier, J., Wiecki, T. V., & Fonnesbeck, C. (2016). Probabilistic programming in Python using PyMC3. *PeerJ Computer Science*, 2, e55.

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* (pp. 8024–8035).

Fey, M., & Lenssen, J. J. (2019). Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2), 257–286.